

Day 17 — BLE GATT Services

Goal

Expose sensor data over Bluetooth Low Energy using a custom GATT service with readable and notifiable characteristics, so a phone can subscribe to live updates.

What you will learn

- The difference between advertising (Day 18) and GATT services
- How to define a custom GATT service and characteristics in Zephyr
- How to send notifications when data changes
- How to read a characteristic value from nRF Connect on a phone

Overview

GATT (Generic Attribute Profile) is the data layer of BLE. A **service** groups related **characteristics**. Each characteristic has a value, permissions (read/write/notify), and optional descriptors. Zephyr's `BT_GATT_SERVICE_DEFINE` macro registers a static service at compile time.

Custom service UUIDs

Generate your own 128-bit UUIDs (use [uuidgenerator.net](https://www.uuidgenerator.net/)):

```
/* Service UUID */
#define BT_UUID_SENSOR_VAL \
    BT_UUID_128_ENCODE(0x12345678, 0x1234, 0x5678, 0x1234, \
0x56789abcdef0)
#define BT_UUID_SENSOR
BT_UUID_DECLARE_128(BT_UUID_SENSOR_VAL)

/* Accelerometer characteristic UUID */
#define BT_UUID_ACCEL_VAL \
    BT_UUID_128_ENCODE(0x12345678, 0x1234, 0x5678, 0x1234,
```

```
0x56789abcdef1)
```

```
#define BT_UUID_ACCEL
```

```
BT_UUID_DECLARE_128(BT_UUID_ACCEL_VAL)
```

Defining the GATT service

```
#include <zephyr/bluetooth/gatt.h>

static int16_t accel_data[3] = {0, 0, 1000}; /* X, Y, Z in mg */

static ssize_t read_accel(struct bt_conn *conn,
                          const struct bt_gatt_attr *attr,
                          void *buf, uint16_t len, uint16_t
offset)
{
    return bt_gatt_attr_read(conn, attr, buf, len, offset,
                             accel_data, sizeof(accel_data));
}

BT_GATT_SERVICE_DEFINE(sensor_svc,
    BT_GATT_PRIMARY_SERVICE(BT_UUID_SENSOR),
    BT_GATT_CHARACTERISTIC(BT_UUID_ACCEL,
        BT_GATT_CHRC_READ | BT_GATT_CHRC_NOTIFY,
        BT_GATT_PERM_READ,
        read_accel, NULL, accel_data),
    BT_GATT_CCC(NULL, BT_GATT_PERM_READ | BT_GATT_PERM_WRITE),
    );
```

Sending notifications

```
void notify_accel(int16_t x, int16_t y, int16_t z)
{
    accel_data[0] = x;
    accel_data[1] = y;
    accel_data[2] = z;
    bt_gatt_notify(NULL, &sensor_svc.attrs[1], accel_data,
sizeof(accel_data));
}
```

prj.conf

```
CONFIG_BT=y
CONFIG_BT_PERIPHERAL=y
CONFIG_BT_DEVICE_NAME="ZephyrSensor"
CONFIG_BT_GATT_DYNAMIC_DB=n
CONFIG_LOG=y
```

Full application flow

```
int main(void)
{
    int ret = bt_enable(NULL);
    if (ret < 0) {
        LOG_ERR("BT enable failed: %d", ret);
        return ret;
    }
    bt_le_adv_start(BT_LE_ADV_CONN, ad, ARRAY_SIZE(ad), sd,
        ARRAY_SIZE(sd));

    while (1) {
        notify_accel(read_x(), read_y(), read_z());
        k_msleep(200);
    }
}
```

Reading from a phone

1. Open **nRF Connect** (Android/iOS) → scan → connect to “ZephyrSensor”
2. Expand the **Unknown Service** (your custom UUID)
3. Tap the **read** button on the characteristic → see raw bytes
4. Tap the **notify** button (bell icon) → live updates appear as data changes

Decoding the characteristic value

The value is 6 bytes — three `int16_t` values in little-endian order.

```
import struct
raw = bytes.fromhex("F4FF2200E803")
x, y, z = struct.unpack('<hhh', raw)
print(f"X={x} Y={y} Z={z} mg")    # X=-12 Y=34 Z=1000 mg
```

Why this matters

GATT services are the foundation of every BLE product — heart rate monitors, fitness trackers, environmental sensors, and industrial IoT all use this pattern. Once you define a service, any BLE central (phone, PC, gateway) can read or subscribe to your data without a custom app.

Practice tasks

1. Flash and connect with nRF Connect — confirm the service appears and read returns data.
2. Enable notifications and shake the board — confirm live X/Y/Z updates on the phone.
3. Add a second characteristic for the measurement range (single byte, read-only).
4. Add a **write** characteristic that lets the phone set the accelerometer range.

Example folder

View the complete source code on GitHub: [src/2026-06-01_BLE_GATT_Service](#)

Next topic

Day 18 covers BLE peripheral advertising in depth and handling connect/disconnect events gracefully.